

深圳大学实验报告

课程名称： 算法设计与分析

实验项目名称： 图论——最大流应用问题

学院： 计算机与软件学院

专业： 计算机科学与技术

指导教师： 刘刚

报告人： 汉雨 学号： 2024040042

实验时间： 2026年6月 日—2026年6月 日

实验报告提交时间： 2026年6月 日

教务部制

一、实验目的：

1. 掌握最大流算法思想。
2. 学会用最大流算法求解应用问题。

二、实验内容：



图 1 《旧金山纪事报》中关于棒球淘汰问题的报道背景

说明：该报道给出了棒球淘汰问题的现实背景，引出如何用最大流判断球队是否仍有夺冠可能。

1996 年 9 月 10 日，《旧金山纪事报》的体育版上登载了《巨人队正式告别 NL 西区比赛》一文，宣布了旧金山巨人队输掉比赛的消息。当时，圣地亚哥教士队凭借 80 场胜利暂列西区比赛第一，旧金山巨人队只赢得了 59 场比赛，要想追上圣地亚哥教士队，至少还得再赢 21 场比赛才行。然而，根据赛程安排，巨人队只剩下 20 场比赛没打了，因而彻底与冠军无缘（摘自 <http://www.matrix67.com/blog/archives/5190>）。

有趣的是，报社可能没有发现，其实在两天以前，也就是 1996 年 9 月 8 日，巨人队就已经没有夺冠的可能了。那一天，圣地亚哥教士队还只有 78 场胜利，与洛杉矶道奇队暂时并列第一。此时的巨人队仍然是 59 场胜利，但还有 22 场比赛没打。因而，表面上看起来，巨人队似乎仍有夺冠的可能。然而，根据赛程安排，圣地亚哥教士队和洛杉矶道奇队互相之间还有 7 场比赛要打，其中必有一方会获得至少 4 场胜利，从而拿到 82 胜的总分；即使巨人队剩下的 22 场比赛全胜，也只能得到 81 胜。由此可见，巨人队再怎么努力，也不能获得冠军了。

在美国职业棒球的例行赛中，每个球队都要打 162 场比赛（对手包括但不限于同一分区里的其他队伍，和同一队伍也往往会有多次交手），所胜场数最多者为该分区的冠军；如果有并列第一的情况，则用加赛决出冠军。在比赛过程中，如果我们发现，某支球队无论如何都已经不可能以第一名或者并列第一名的成绩结束比赛，那么这支球队就提前被淘汰了（虽然它还要继续打下去）。从上面的例子中可以看出，发现并且证明一个球

队已经告败，有时并不是一件容易的事。

关于这个事情有一个有趣的故事，下面是一段对话：

“看到上周报纸上关于爱因斯坦的那篇文章了吗？.....有记者请他算出三角比赛的数学公式。你知道，一支球队赢得了很多剩余的比赛，其他球队则赢这个赢了那个。这个比赛到底有多少种可能性？哪个球队更有优势？”

“他到底知道吗？”

“显然他知道的也不多。上周五他选择道奇队没有选巨人队。”

Team i	Wins w_i	Losses l_i	To play r_i	Against = r_{ij}			
				Atl	Phi	NY	Mon
Atlanta	83	71	8	-	1	6	1
Philly	80	79	3	1	-	0	2
New York	78	78	6	6	0	-	0
Montreal	77	82	3	1	2	0	-

图 2 teams4.txt 四支球队原始赛程数据

说明：该图展示了 4 支球队的胜场、负场、剩余场次及相互比赛数，是后续超小规模求解结果表的输入来源。

上面的表是四个球队的比赛情况，现在的问题是哪些球队有机会以最多的胜利结束这个赛季？可以看到蒙特利尔队因最多只能取得 80 场胜利而被淘汰，但亚特兰大队已经取得 83 场胜利，蒙特利尔队因为 $w_i + r_i < w_j$ 而被淘汰。费城队可以赢 83 场，但仍然会被淘汰。。。如果亚特兰大输掉一场比赛，那么其他球队就会赢一场。所以答案不仅取决于已经赢了多少场比赛，还取决于他们的对手是谁。

请利用最大流算法给出上面这个棒球问题的求解方法。

四、实验内容及过程：

(一)流网络的构造原理。

假设要判断球队 X 是否还有机会夺冠。先计算它理论上最多能赢多少场：

$$W_{\max} = w_X + r_X$$

其中： w_X ：球队 X 现在已经赢的场数； r_X ：球队 X 剩余比赛数； $W_{\max}$ ：即使 X 后面全赢，最多能达到的胜场数。

接着，暂时不看 X 自己，只看其他球队之间还没打的比赛。看看若其他球队之间没打的比赛能否在条件的约束下合理的分配完。如果能分配完，就说明这个球队有机会得冠

网络图可以分成以下三个部分：

源点到比赛节点

若球队 A 和 B 还有 g_{AB} 场比赛： $s \rightarrow G_{AB}$ ，意思是：这 g_{AB} 场比赛总共一定会产生 g_{AB} 个胜场，需要全部分配出去。

比赛节点到球队节点

$$G_{AB} \rightarrow A$$

$$G_{AB} \rightarrow B$$

容量设为无穷大，或一个足够大的数。（ A 和 B 的每一场比赛，胜利可以分给 A ，也可以分给 B 。）

例如 A 和 B 还要打 3 场：

- 流量 3 全流向 A ：表示 A 连赢 3 场；
- 流量 2 流向 A 、1 流向 B ：表示 A 赢 2 场， B 赢 1 场。

所以，比赛节点连向球队节点，本质上是在决定每场比赛谁获胜。

球队节点到汇点

$$A \rightarrow t$$

容量设为：

$$W_{\max} - w_A$$

为了不超过 X 的理论最高胜场 $W_{\max}$ ，球队 A 最多还能被分到多少场胜利。

最终判断 X 是否被淘汰

设所有“其他球队之间未进行比赛”的总场数为：

$$G = \sum g_{ij}$$

然后求网络最大流。

若最大流 $= G$ ，说明所有剩余比赛的胜场都能合理分配，又不会让任何球队超过 X 的最高胜场。所以 X 还没有被淘汰。

反之则被淘汰。

(二)为什么最大流能解决这个问题。

Ford-Fulkerson 方法解决此问

Ford-Fulkerson 方法的思路很简单。

每次随便找一条从源点 s 到汇点 t 的可行路径，然后尽量多送一些流量过去。然后更新剩余容量，继续找下一条增广路。直到再也找不到从 s 到 t 的路径。

最后，把得到的最大流与总比赛数 G 比较，就能判断球队 X 是否淘汰。我们用简单的例子说明 Ford-Fulkerson 方法：

目标球队目前最多可以胜利 10 场，其他球队之间剩余场次（未分配场次）是 5 场。一下过程左边为最大流网络图，右边是残缺网络图。

初始：

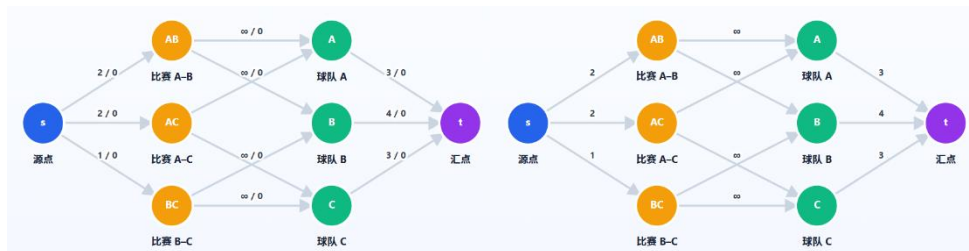


图 3 Ford-Fulkerson 初始流网络与残量网络

说明：左侧表示当前流网络，右侧表示对应残量网络，后续增广都在残量网络中寻找可行路径。

第 1 次增广：给 A-B 的两场比赛都分配给 A。

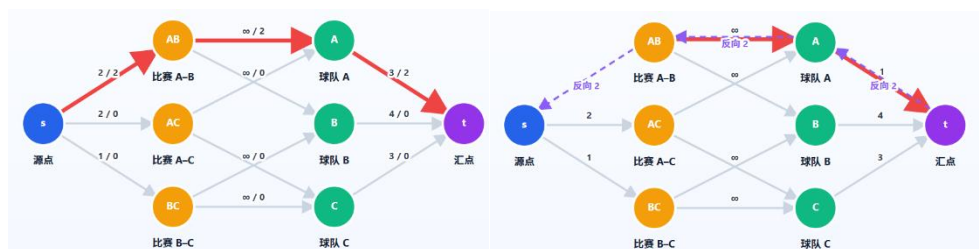


图 4 Ford-Fulkerson 第 1 次增广

第 2 次增广：给 A-C 的两场比赛都分配给 C。

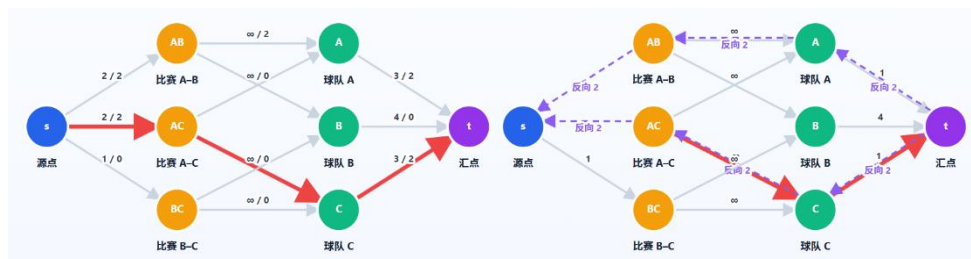


图 5 Ford-Fulkerson 第 2 次增广过程

第 3 次增广：给 B-C 的一场比赛分配给 B。

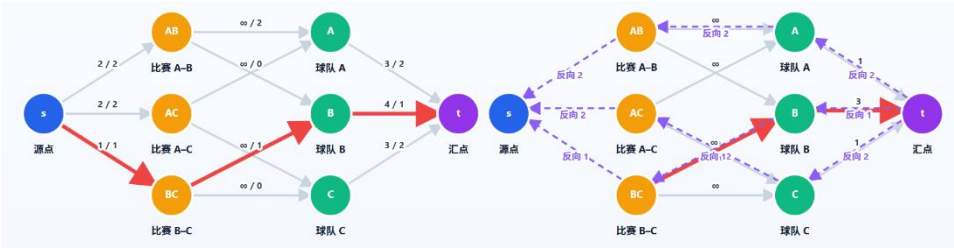


图 6 Ford-Fulkerson 第 3 次增广过程

说明：继续沿残量网络寻找可行路径，将 B-C 的剩余比赛胜场分配到可行球队节点。

结束：最大流 = 5，等于全部剩余比赛数

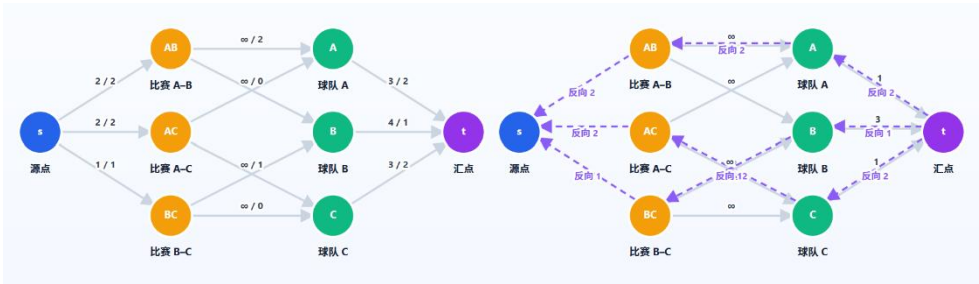


图 7 最大流达到全部剩余比赛数的结束状态

说明：当最大流等于所有待分配比赛数时，说明存在一种胜场分配方案使目标球队不被淘汰。

算法 3 Ford-Fulkerson 算法

输入： 流网络 G ，源点 s ，汇点 t

输出： 最大流值 $flow$

```
1:  $flow \leftarrow 0$ 
2: loop
3:   清空访问标记
4:    $pushed \leftarrow \text{DFS}(s, t, INF)$ 
5:   if  $pushed = 0$  then
6:     break
7:   end if
8:    $flow \leftarrow flow + pushed$ 
9: end loop
10: return  $flow$ 
11: procedure  $\text{DFS}(v, t, available)$ 
12:   if  $v = t$  then
13:     return  $available$ 
14:   end if
15:   标记  $v$  已访问
16:   for 每条残量边  $e = (v, to)$  do
17:     if  $e.cap > 0$  且  $to$  未访问 then
18:        $pushed \leftarrow \text{DFS}(to, t, \min(available, e.cap))$ 
19:       if  $pushed > 0$  then
20:         更新正向边和反向边残量
21:         return  $pushed$ 
22:       end if
23:     end if
24:   end for
25:   return 0
26: end procedure
```

伪代码说明：该流程展示 Ford-Fulkerson 反复寻找增广路、按瓶颈容量推流并更新残量网络的基本循环。

时间复杂度分析

Ford-Fulkerson 算法若网络中的边容量均为整数，则每次增广至少能使总流量增加 1，因此最多需要进行 $(|f^*|)$ 次增广，其中 $(|f^*|)$ 表示最大流的数值。每次寻找增广路径时，最坏情况下需要遍历图中的全部边，时间复杂度为 $(O(E))$ ，因此 Ford-Fulkerson 算法的总时间复杂度为 $(O(E \cdot |f^*|))$ 。该复杂度与最大流的具体数值有关，当边容量较大、最大流值较大时，算法可能需要进行较多轮增广，运行效率会明显下降。



Edmonds-Karp 算法

Edmonds-Karp 算法可以看作 Ford-Fulkerson 的确定化版本。它仍然使用残量网络和增广路，使用 BFS 找到从 s 到 t 的最短增广路。这里的“最短”指边数最少

这种选择带来了严格的复杂度保证。直观地说，BFS 使得源点到每个点的距离层次不会反复下降，每条边成为关键瓶颈的次数受到限制，因此总增广次数不再依赖最大流值 F ，而被限制在 $O(N_f M_f)$ 量级。每次 BFS 扫描 $O(M_f)$ 条边，所以总复杂度为

$$O(N_f M_f^2).$$

在本程序实现中，BFS 队列保存待扩展节点，数组 `parentV` 与 `parentE` 记录每个节点的前驱节点与前驱边编号。找到汇点后沿前驱数组回溯路径，先求瓶颈容量，再统一更新正向边和反向边容量。

与 Ford-Fulkerson 相比，Edmonds-Karp 的重要特点是复杂度不直接依赖最大流值 F ，而是由流网络的顶点数 N_f 和边数 M_f 给出确定上界。在本实验的密集成赛赛程中， N_f 、 M_f 都会随着球队数快速增长，所以 Edmonds-Karp 的曲线仍会明显上升；但由于 BFS 避免了 DFS 任意选择增广路带来的低效路径，本实验结果显示它在相同规模下明显快于 Ford-Fulkerson。

算法 4 Edmonds-Karp 算法

输入： 流网络 G ，源点 s ，汇点 t

输出： 最大流值 $flow$

```
1:  $flow \leftarrow 0$ 
2: while BFS 能在残量网络中找到  $s$  到  $t$  的路径 do
3:   根据前驱数组恢复增广路径  $P$ 
4:    $bottleneck \leftarrow \min\{residual(e) \mid e \in P\}$ 
5:   沿路径  $P$  增广  $bottleneck$  单位流量
6:    $flow \leftarrow flow + bottleneck$ 
7: end while
8: return  $flow$ 
```

伪代码说明：该流程展示 Edmonds-Karp 使用 BFS 固定最短增广路，并通过前驱数组回溯更新残量网络。

(三)优化的最大流算法。

Dinic 方法解决此问

Dinic 算法也是求**最大流**的。它不会每找到一条路径就立刻重新找下一条，而是先看一眼：**现在从 S 到各个点，最短要走几步？**

比如 BFS 后发现：S 在第 0 层；A、B 在第 1 层；C、D 在第 2 层；T 在第 3 层
那么 Dinic 只允许流量按下面这种方向走：

$$level[v] = level[u] + 1$$

这种按层数往前走的网络，叫做**分层图**。有了分层图以后，Dinic 不马上重新 BFS。它会在这张分层图里不断用 DFS 找路径、推流。

例如先找到： $S \rightarrow A \rightarrow C \rightarrow T$

假设这条路最多能送 2 单位流，就先送 2。然后它不会停，而是继续在**同一张分层图**里找别的路，比如： $S \rightarrow B \rightarrow D \rightarrow T$ 又送 3 单位。

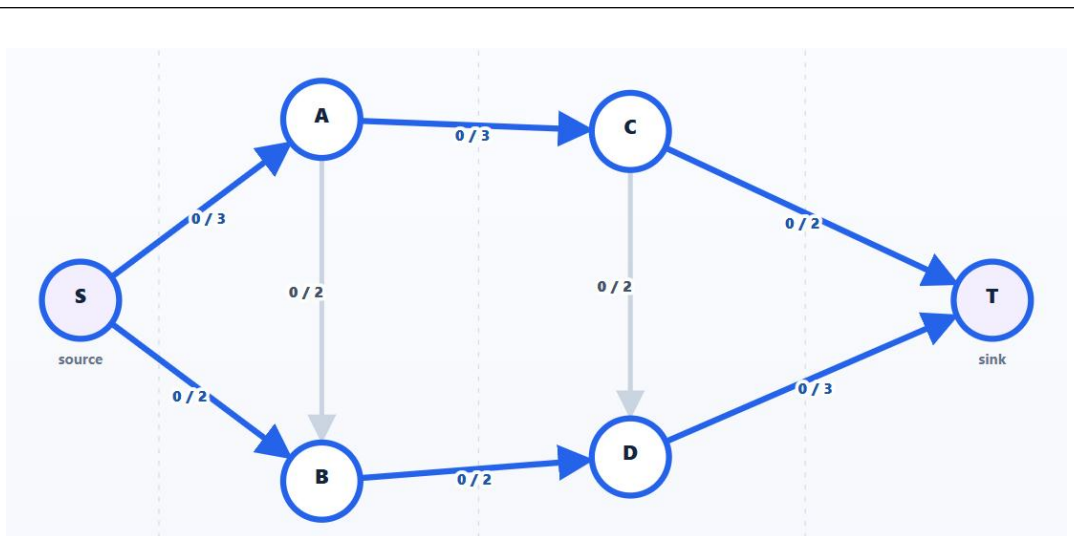


图 8 Dinic 分层图中的可行推进方向

说明：BFS 建立层次后，DFS 只沿层数递增的边推流，避免无意义回退。

直到出现一种情况：从 s 出发，已经找不到任何一条能沿层数一直走到 t 的路径这就叫这一轮形成了**阻塞流**。这时才重新 BFS，重新计算层数。

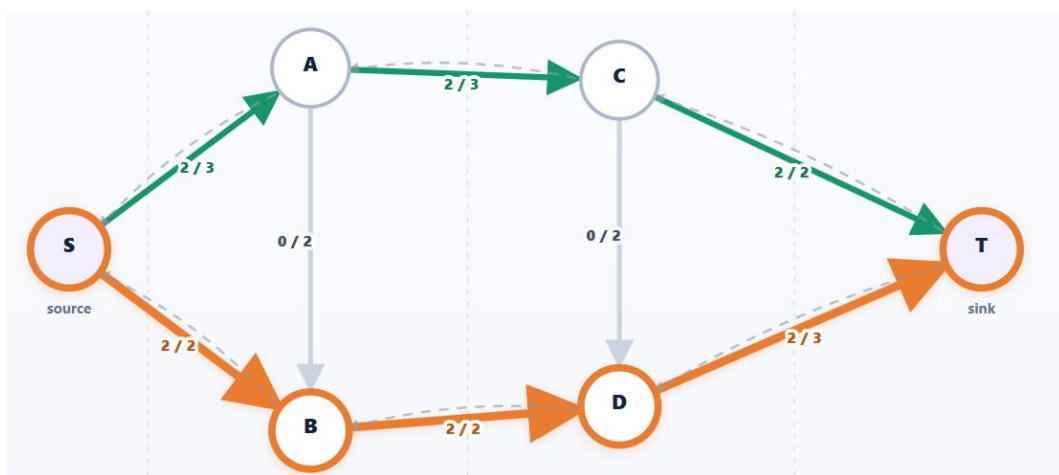


图 9 Dinic 在同一层次图中的阻塞流推送

说明：同一轮 BFS 后可以连续寻找多条可行增广路，直到该层次图无法继续推流。

因为前面推流后，有些边容量被用满了，残量网络变了。原来三步能到 T 的路径，可能断掉了；现在可能要绕四步、五步才能到 T 。

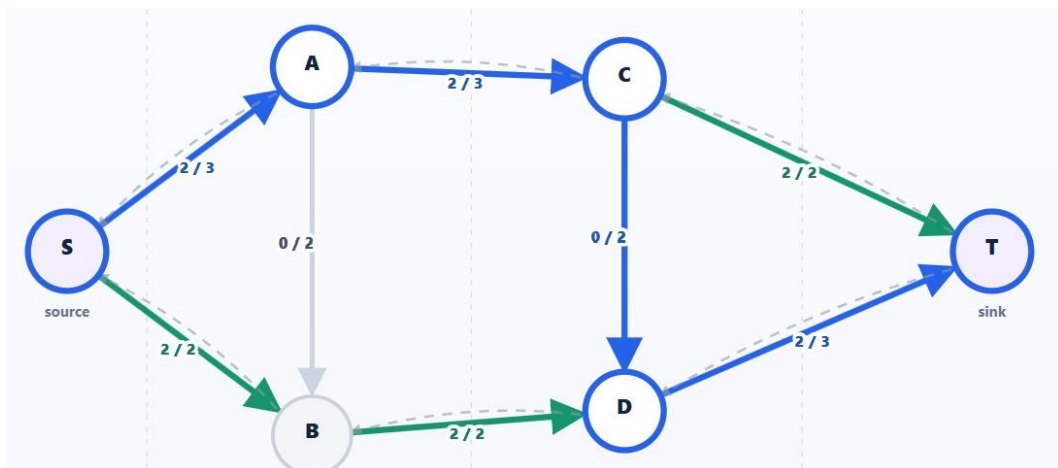


图 10 残量网络变化后的重新分层

说明：部分边被推满后，残量网络结构改变，需要重新 BFS 计算层次。

例如原来： $S \rightarrow A \rightarrow C \rightarrow T$ 是 3 条边。但 $C \rightarrow T$ 满了以后，可能只能走： $S \rightarrow A \rightarrow C \rightarrow D \rightarrow T$ 变成 4 条边。

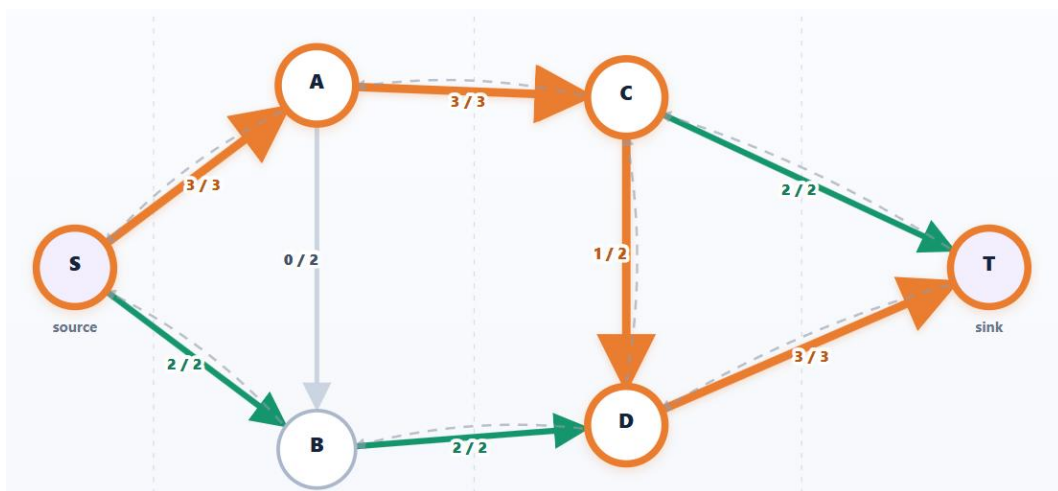


图 11 更长增广路径出现时的层次变化

说明：原短路径被阻断后，算法会转向新的更长路径继续寻找可行流。

BFS 连 T 都到不了时，最大流完成

Ford 每次只关心“找到一条能走的路”；Dinic 会先把“这一批最短可走的路”全找出来，一口气尽量推完。

Ford 的选路很随意，可能路径会很绕；

Dinic 强制按层推进，不走回头路、不走同层边；Dinic 一轮 BFS 后，会把这一层里能送的流尽量全部送完，减少反复搜索。

Dinic 算法将增广过程划分为多个阶段：每个阶段先利用 BFS 在残量网络中建立层次图，再利用 DFS 在层次图中寻找增广路径并持续推送流量，直到当前层次图中无

法继续增广，形成一个阻塞流。由于每完成一个阶段，源点到汇点的最短增广路径长度至少增加 1，因此阶段数最多为 $(V-1)$ 次，其中 (V) 为顶点数。在未使用当前弧优化的情况下，DFS 在多次寻找增广路径时可能反复从同一顶点的第一条边开始检查，导致已经确认无法继续推流的边被重复访问，单个阶段的最坏时间复杂度可达到 $(O(E^2))$ 。

因此，该朴素实现下 Dinic 算法的总时间复杂度通常记为 $(O(VE^2))$ 。

算法 5 Dinic 算法

输入： 流网络 G ，源点 s ，汇点 t

输出： 最大流值 $flow$

```
1:  $flow \leftarrow 0$ 
2: while BFSLEVELGRAPH( $s, t$ ) 成功 do
3:   对每个顶点  $v$ ，令当前弧指针  $it[v] \leftarrow 0$ 
4:   loop
5:      $pushed \leftarrow$  DFSBLOCKINGFLOW( $s, t, INF$ )
6:     if  $pushed = 0$  then
7:       break
8:     end if
9:      $flow \leftarrow flow + pushed$ 
10:  end loop
11: end while
12: return  $flow$ 
13: procedure DFSBLOCKINGFLOW( $v, t, available$ )
14:   if  $v = t$  then
15:     return  $available$ 
16:   end if
17:   for 从当前弧  $it[v]$  开始枚举边  $e$  do
18:     if  $e.cap > 0$  且  $level[e.to] = level[v] + 1$  then
19:        $pushed \leftarrow$  DFSBLOCKINGFLOW( $e.to, t, \min(available, e.cap)$ )
20:       if  $pushed > 0$  then
21:         更新正向边和反向边残量
22:         return  $pushed$ 
23:       end if
24:     end if
25:   end for
26:   return 0
27: end procedure
```

伪代码说明：该流程展示 Dinic 先用 BFS 建层，再在层次图中通过 DFS 寻找阻塞流的核心步骤。

Dinic + 当前弧优化

就是在 Dinic 算法找增广路时，避免每次都从一条边的开头重新试。先回忆 Dinic 每一轮做两件事：BFS 建立分层图。DFS 在分层图中不断找增广路，直到这一层图再也推不出流量 问题在于：DFS 可能会反复访问一些“已经证明走不通”的边。

当前弧优化的核心

给每个点 u 设置一个指针： $cur[u]$ 它表示：从 u 的第几条邻边开始继续找。第一次访问 u 时： $cur[u] = 0$ 如果第 0 条边不行，就让： $cur[u]++$ 。下次再来到 u ，直接从下一条边开始，不再回头检查刚才失败的边。

图中红框区域为“断路”。普通 DFS 会在每一次回溯的时候都“不长记性的”走到“断路”里面。但如果是有当前弧优化，就会将 $cur[s]$ 设置为 1，意味着下一次从下一条分支开始走。

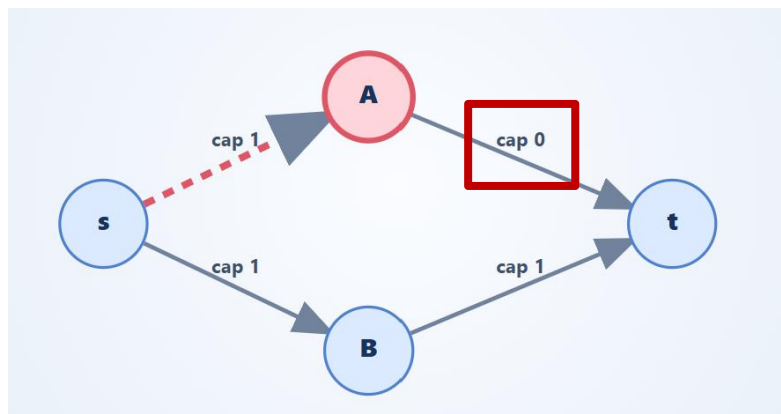


图 12 当前弧优化跳过失效分支示意

说明：当前弧指针会记住已经失败的边，下次 DFS 直接从下一条边开始搜索，减少重复检查。

在 Dinic 算法中加入当前弧优化后，每个顶点都会记录当前已经检查到的边的位置。当某条边已经被证明无法继续推流、容量不足或不满足层次图要求时，后续 DFS 不会再次从该顶点的第一条边重新检查，而是直接从记录的位置继续向后搜索。这样可以避免在同一个层次图中反复访问失败边，使每条边在一个阶段内只会被有限次检查。结合 BFS 建立层次图和 DFS 寻找阻塞流的过程，每个阶段的时间复杂度可控制在 $(O(VE))$ ，而阶段数最多为 $(O(V))$ ，因此加入当前弧优化后的 Dinic 算法总时间复杂度为 $(O(V^2E))$ 。该优化显著减少了无效搜索，是 Dinic 算法实际实现中最常用、最重要的优化之一。

(四)四个球队的求解结果。

表 1 teams4.txt 四支球队求解结果

球队	当前胜场	剩余场次	最高可能胜场	求解结果
Atlanta	83	8	91	未淘汰
Philadelphia	80	3	83	已淘汰
New_York	78	6	84	未淘汰
Montreal	77	3	80	已淘汰

结论：该表将超小规模样例由图片化数据改为可编辑表格，能直观看出 Montreal 属于简单淘汰，Philadelphia 属于非平凡淘汰。

(五)较大数据规模下的实验：

数据来源：<https://coursera.cs.princeton.edu/algs4/assignments/baseball/specification.php>

中 Baseball Elimination 项目的 teams*.txt 测试数据；本实验选取 n=4、10、24、36、48、60 六种规模。计时阈值设为 5 秒，超过阈值记为未能在有限时间内完成。

表 2 较大规模数据下四种最大流算法运行时间

数据集	Ford-Fulkerson	Edmonds-Karp	Dinic	Dinic + 当前弧优化
teams4.txt	0.05 ms	0.06 ms	0.07 ms	0.06 ms
teams10.txt	1.36 ms	1.59 ms	0.88 ms	0.85 ms
teams24.txt	80.6 ms	60.7 ms	22.2 ms	18.4 ms
teams36.txt	148.0 ms	133.3 ms	105.0 ms	106.7 ms
teams48.txt	176.5 ms	177.3 ms	136.2 ms	127.5 ms
teams60.txt	223.6 ms	208.7 ms	172.3 ms	177.9 ms

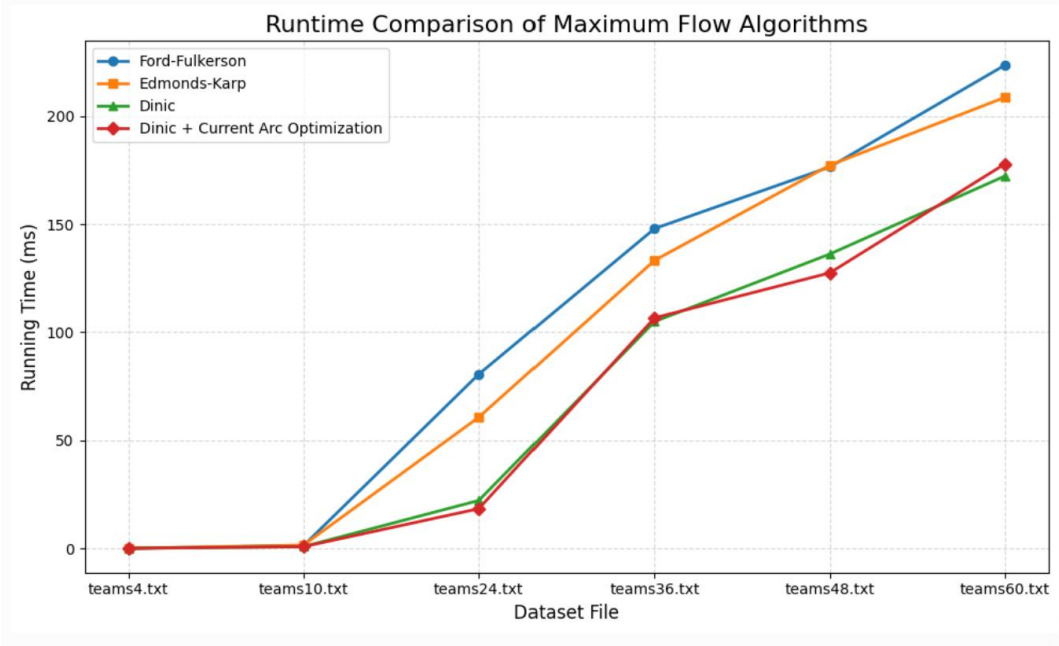


图 13 四种最大流算法运行时间对比曲线

五、实验结果及分析：

结果观察：随着数据规模增大，四种算法的运行时间均明显上升。小规模数据集下差距较小；从 teams24.txt 开始，Dinic 及其当前弧优化版本整体明显快于 Ford-Fulkerson 和 Edmonds-Karp，说明分层图和阻塞流机制能够减少重复寻找增广路径的次数。

Ford-Fulkerson 在大规模数据集上耗时最高，Edmonds-Karp 相对更稳定但仍需多次 BFS。Dinic 在多数测试中效率最好；加入当前弧优化后，在 teams10.txt、teams24.txt 和 teams48.txt 中进一步降低了运行时间，但在个别数据集上略慢于普通 Dinic，可能是当前弧维护开销及测试波动造成的。总体来看，**Dinic 类算法更适合较大规模的网络流问题。**

六、实验结论：

本实验以棒球淘汰问题为背景，完成了流网络模型的构建，并分别采用 Ford-Fulkerson、Edmonds-Karp、Dinic 以及 Dinic 加当前弧优化四种最大流算法进行求解。实验结果表明，最大流方法能够有效判断某支球队是否仍存在夺冠或并列第一的可能：当最大流等于其他球队之间所有待进行比赛的总场数时，说明剩余比赛的胜场可以合理分配，目标球队未被淘汰；反之则说明至少有其他球队必然超过目标球队的最高可能胜场，目标球队已被淘汰。

在 teams4.txt 样例中，算法正确判断 Atlanta 和 New_York 未被淘汰，Montreal 属于由最高可能胜场不足导致的简单淘汰，Philadelphia 则属于由其他球队之间剩余比赛安排导致的非平凡淘汰，验证了流网络模型和算法实现的正确性。性能测试表明，随着球队数量增加，四种算法的运行时间均有所上升。其中，Ford-Fulkerson 的增广路径选择具有较大随机性，整体效率较低；Edmonds-Karp 通过 BFS 选择最短增广路，运行更加稳定；Dinic 算法利用分层图和阻塞流机制减少了重复搜索，在较大规模数据上总体表现更好。加入当前弧优化后，部分数据集的运行时间进一步降低，但在少数测试中受维护开销和测试波动影响，未必始终优于普通 Dinic。

综上，最大流算法能够将实际赛程约束转化为可计算的网路流问题，具有清晰的建模思路和良好的通用性。在本实验的测试范围内，Dinic 及其当前弧优化版本更适合处理规模较大的棒球淘汰问题。



指导教师批阅意见:

成绩评定:

指导教师签字:

2025 年 月 日

备注:

注：1、报告内的项目或内容设置，可根据实际情况加以调整和补充。

2、教师批改学生实验报告时间应在学生提交实验报告时间后 10 日内。